

Randarea Apei folosind Metoda Sumei Sinusurilor

Tema

Tema acestui proiect este realizarea unei scene de ocean folosind metoda Sumei Sinusurilor ("Sum of Sines") într-o manieră ușor de implementat și de rulat într-un joc video.

Motivație

Randarea apei este un aspect important pentru domeniul de Real Time Rendering, fiind regăsită atât în industria jocurilor video cât și cea cinematografică. Deși există metode științifice pentru simularea apei bazate pe Ecuațiile Navier-Stokes, acele metode nu sunt fezabile într-un context în timp real din cauza complexității acestora.

Obiective propuse

Cameră ce permite mișcarea în jurul scenei

Randarea apei

Realizarea unui skybox

Reflexii pentru apă folosind skybox-ul

Realizarea unui model complex ce plutește pe apă

Mediu de lucru

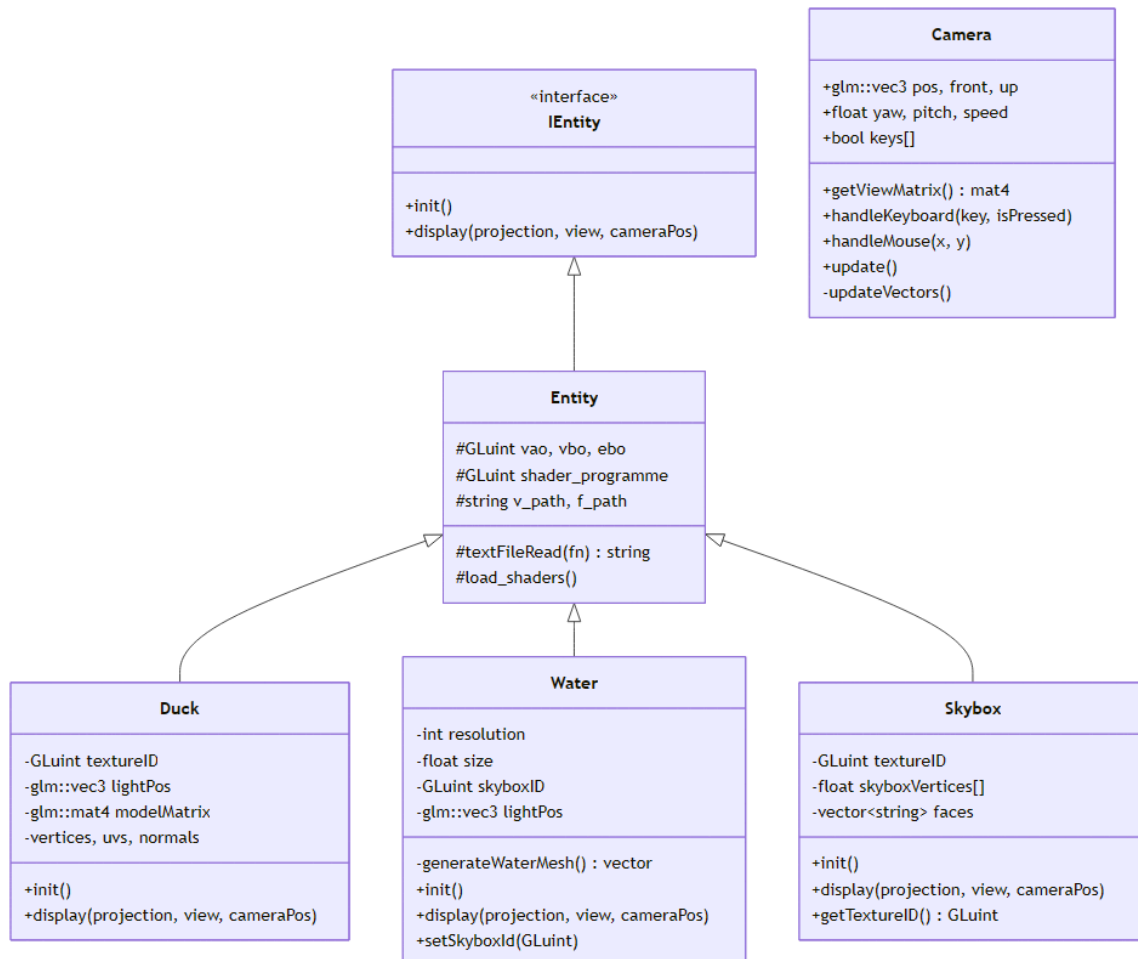
- Visual Studio C++ 14
- GLSL vers 400
- FreeGlut

Biblioteci

- stb_image pentru încărcarea imaginilor folosite la texturi
- objloader pentru încărcarea modelului
- miniaudio pentru sunet ambiental de ocean

Metodologie

Pentru a gestiona mai ușor fiecare obiect al scenei am creat o clasă abstractă de bază numită Entity ce este moștenită de fiecare obiect (apă, skybox, model).



Camera Scenei și controale

Camera permite mișcarea liberă în scenă folosind tastele WASD pentru deplasare și mouse-ul pentru rotație. Orientarea este calculată din unghiurile yaw (stânga-dreapta) și pitch (sus-jos). Tastele Q și E deplasează camera în sus și în jos iar tasta ESC închide programul.

Generarea valurilor

Există 2 metode populare de a realiza acest lucru. Prima metodă este suma sinusurilor ("Sum of Sines") ce constă în modificarea coordonatei Y a apei în funcție de rezultatul sumei. A doua metodă este folosirea unui spectru de frecvențe și transformata Fourier rapidă ("FFT").

Datorită complexității de implementare a celei de-a doua metode s-a decis folosirea sumei sinusurilor pentru realizarea acestui proiect.

Apă

Pentru generarea mesh-ului de apă este folosită funcția `generateWaterMesh()`. Aceasta creează un plan orizontal pe axa XZ centrat în origine. Resolution este numărul de pătrate (pe o axă) iar `squareSize` este mărimea unui pătrat. Funcția iterează prin numărul de pătrate, calculează componentele x și z pentru un pătrat și după le adaugă într-un vector de float.

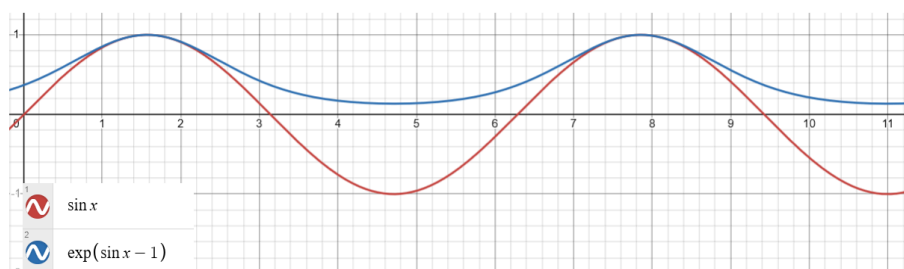
Uniformele trimise către shader sunt time pentru animația apei, textura cubemap a skybox-ului pentru reflexii, matricea mvp și poziția luminii și a camerei.

Vertex shader

Scopul vertex shaderului este de a modifica componenta Y a planului de apă pentru a crea aspectul de val.

Funcția `calculateWave` este cea care se ocupă de acest lucru. Ea primește ca parametri unghiul, lungimea de undă, viteza și amplitudinea și returnează un `vec3` cu înălțimea și derivatele parțiale pe x și z ce vor fi folosite.

În loc de o sinusoidă simplă s-a ales funcția $\exp(\sin(\text{phase}) - 1.0)$ ce are un aspect mai apropiat de cel al valurilor reale.



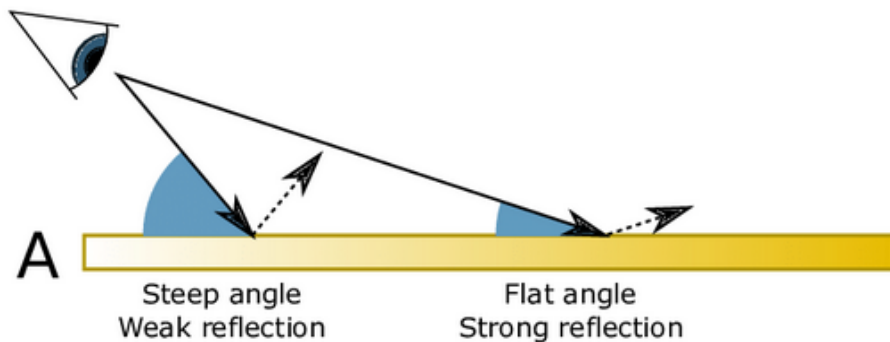
Pentru a obține rezultatul final s-a aplicat algoritmul fractional brownian motion. Acest algoritm presupune aplicarea unor scalări asupra parametrilor valurilor în timpul calculului sumei. Parametrii de scalare cât și parametri inițiali ai valurilor au fost aleși empiric.

După calculul acestei sume se modifică componenta Y, normalele sunt recalculat și trimise către fragment shader.

Fragment shader

Fragment shader-ul implementează iluminarea Phong la care am adăugat reflexiile obținute din skybox cât și componenta Fresnel.

Efectul Fresnel reprezintă fenomenul în care suprafețele văzute la unghiuri mici față de privitor reflectă mult mai multă lumină decât cele văzute la unghiuri mari.



Componenta speculară este înmulțită cu $(0.2 + 0.8 * \text{fresnel})$ pentru a avea mereu un minim de 20% speculară, aceasta crescând până la 100% la unghiuri mari. Culoarea finală este un mix de $0.2 + \text{fresnel}$ între culoarea de bază din modelul de iluminare și cea a reflexiilor.

Skybox

Skyboxul este un cubemap la care se aplică 6 texturi ce reprezintă cerul. Acesta primește în constructor un vector<String> reprezentând căile către imaginile folosite pentru texturi. În funcția de display folosește doar ViewMatrix pentru a nu fi translatat.

Vertex shaderul are ca rezultat $gl_Position = p.xyww$. La împărțirea z/w se va obține rezultatul 1.0 (maximul) ceea ce face skybox-ul să fie desenat în spatele oricărui obiect din scenă. Acesta este și motivul pentru care $glDepthFunc$ trebuie să aibă valoarea GL_LEQUAL , fără aceasta nu ar fi desenat deoarece valoarea implicită GL_LESS ar interzice asta.

Rața

Rața reprezintă modelul complex al scenei. Modelul a fost preluat de pe free3d.com și conține valori pentru poziție, normale și coordonatele UV ale texturii. Încărcarea sa este făcută folosind objloader modificat pentru a putea lua coordonatele UV.

Aceasta este animată prin 4 transformări aplicate model matrix-ului. Prima este o translație pe axa Y pentru a simula plutirea pe apă. Următoarele 3 sunt rotații: o balansare stânga-dreapta și o înclinare față-spate, ambele animate cu funcția $\sin(\text{time})$ pentru a simula mișcarea pe valuri, și o rotație continuă în jurul axei Z.

Rezultate



Rezultatul obținut este o scenă de ocean credibilă și eficientă din punct de vedere al performanței. Metoda prezintă însă limitări vizibile: privită de sus se observă tilingul mesh-ului, iar alegerea parametrilor undelor este un proces empiric dificil de controlat.

Concluzii

Metoda sumei sinusurilor reprezintă o soluție accesibilă cu rezultate acceptabile pentru randarea apei în timp real. Din cauza limitărilor sale aceasta este potrivită pentru ape stătătoare de mărimi medii sau în scene unde oceanul nu are un rol important.

În cazurile în care este cerută o fidelitate crescută, metoda bazată pe spectre oceanice, de exemplu spectrul JONSWAP și transformata Fourier rapidă, produce rezultate mai realiste, plăcute vizual și cu un grad mare de parametrizare în schimbul unei complexități de implementare mai înalte.

Potențiale Îmbunătățiri pe viitor

- Înlocuirea metodei sum of sines cu FFT
- Animația raței să urmeze înălțimea apei
- Detectarea spumei folosind operatorul Jacobian și acumularea acestuia într-o textură
- Trecerea de la iluminare Phong la o iluminare bazată pe fizică (PBR)
- Antialiasing
- Efecte de post-procesare (HDR, bloom)

Referințe Bibliografice

Referințe teoretice

<https://learnopengl.com/Lighting/Basic-Lighting> : pentru iluminare

<https://learnopengl.com/Advanced-OpenGL/Cubemaps> : pentru skybox

<https://www.youtube.com/watch?v=PH9q0HNBjT4&t=831s> : teoria din spate

https://www.wikiwaves.org/index.php/Ocean-Wave_Spectra : spectre oceanice

Asset-uri

<https://bumbadida.itch.io/skybox-textures-sdr> : skybox

https://free3d.com/3d-model/bird-v1--282209.html?dd_referrer= : model rata

<https://skyboxgen.com/> : generator skybox

<https://pixabay.com/sound-effects/nature-soothing-ocean-waves-372489/> : sunet ocean